2026 edition

Deep Learning for Music Analysis and Generation

Synthesis & MIDI-to-Audio Generation

(midi → audio; X → audio)



Yi-Hsuan Yang Ph.D.
yhyangtw@ntu.edu.tw

Reference: ISMIR 2022 Tutorial

https://github.com/lukewys/ISMIR2022-tutorial

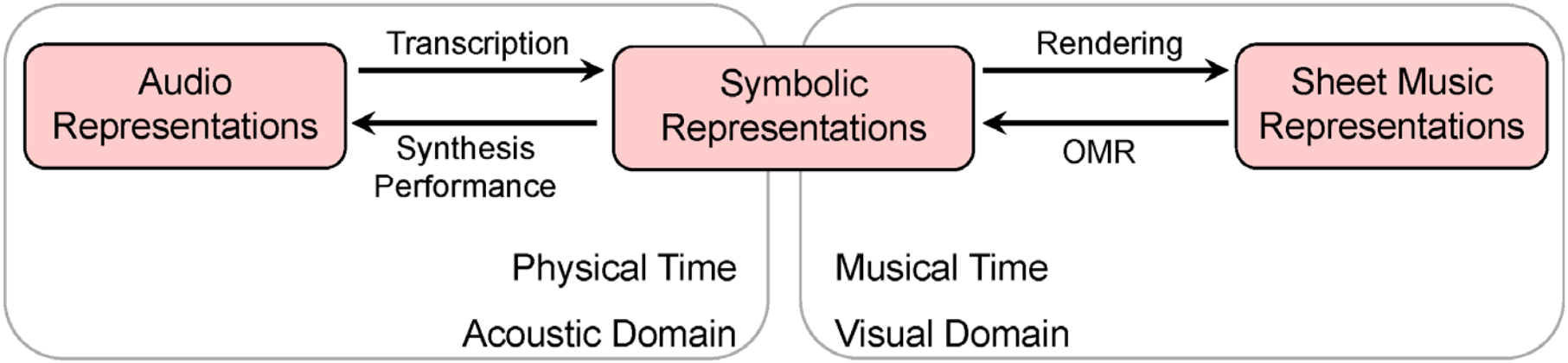
https://youtu.be/7U-zDL5con8?si=HcD7YDN66YPlyGCN&t=9783





Data Representation of Music

(Figure source: https://www.mdpi.com/2624-6120/2/2/18)



- **Musical audio** (waveforms, spectrograms)
  - what we "hear"
  - involve sounds
- **Symbolic representations** (e.g., MIDI, MusicXML)
  - the way music can be represented in a Digital Audio Workstation (DAW)
  - still relevant today as it's *interpretable* and *editable*
- **Sheet music** (visual representations of a musical score)
  - what musicians "compose"

MIDI-to-Audio Generation

- **Note-by-note** (synthesizers)
  - Take pitch and duration as condition
  - Generate single notes

- MIDI **sequence** to audio sequence
  - More general than single-note generation

- **Text-to-MIDI-to-music** generation

Outline

- **DSP-based music synthesizers**
- DL-based generation of single notes
- DL-based generation of note sequences
- MIDI-to-audio

Synthesizers

https://en.wikipedia.org/wiki/Synthesizer

- An electronic musical instrument that generates audio signals
  - **Analog**: circuits, voltage-controlled oscillator

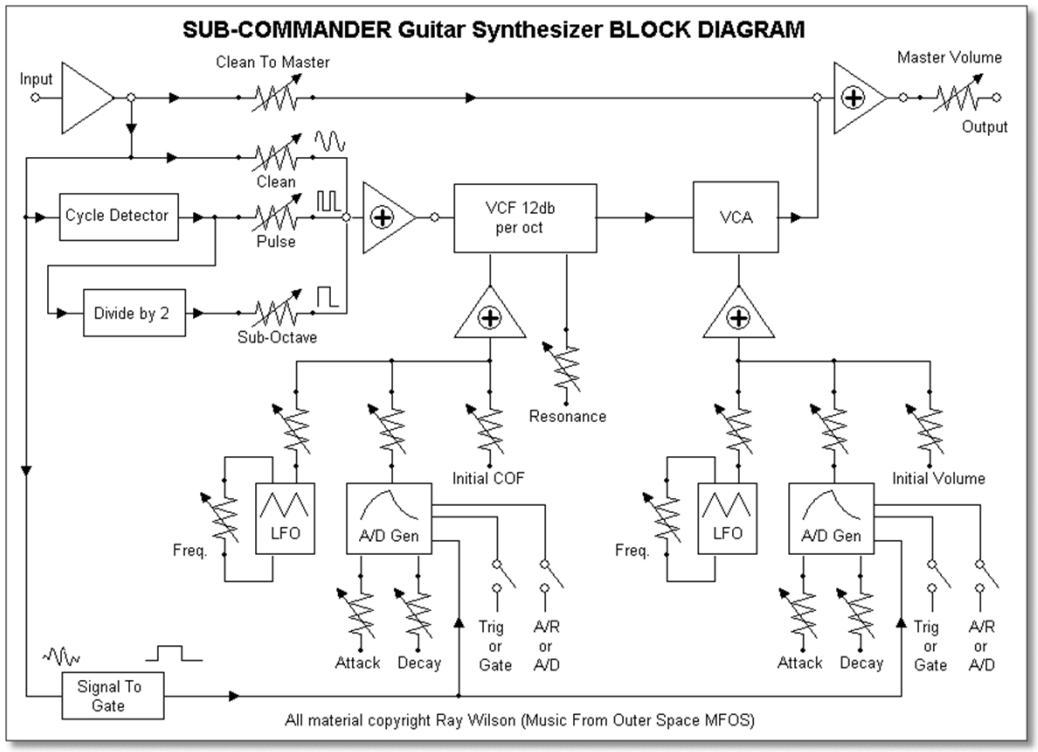


Figure from:
http://musicfromouterspace.com/analogsynth_new/GUITARSYNTHAUG2007/GUITARSYNTHAUG2007.html

Synthesizers

https://en.wikipedia.org/wiki/Synthesizer

- An electronic musical instrument that generates audio signals
  - **Analog**: circuits, voltage-controlled oscillator
  - **Digital**: through DSP
    - Yamaha DX7: based on frequency modulation (FM) synthesis technology





FM Synthesis

https://en.wikipedia.org/wiki/John_Chowning

- John M. Chowning is coming to NTU and give a talk on Dec 4!
  - CSIE auditorium, 14:20

DSP-based Synthesizers

https://en.wikipedia.org/wiki/Synthesizer

- Different types of synthesizers
  - **Subtractive synthesis:** complex waveforms are generated by oscillators and then shaped with filters to remove or boost specific frequencies
  - **Additive synthesis:** a large number of waveforms, usually sine waves, are combined into a composite sound
  - **FM synthesis:** a carrier wave is modulated with the frequency of a modulator wave
  - **Wavetable synthesis:** synthesizers modulate smoothly between digital representations of different waveforms, changing the shape and timbre
  - **Granular synthesis**
  - **Sample-based synthesizer**
  - **Physical modelling synthesis**

DSP-based Synthesizers

https://www.coursera.org/learn/audio-signal-processing

https://github.com/MTG/sms-tools

https://cs.gmu.edu/~sean/book/synthesis/

Week 1: Introduction; basic mathematics
Week 2: Discrete Fourier transform
Week 3: Fourier transform properties
Week 4: Short-time Fourier transform
Week 5: Sinusoidal model
Week 6: Harmonic model
Week 7: Sinusoidal plus residual modeling
Week 8: Sound transformations
Week 9: Sound/music description
Week 10: Concluding topics; beyond audio signal processing

Topics in the Book

1. **Introduction.** Synthesizer basics, usage environments, basic history.
2. **Representations of sound.** Units of measure, issues in digitization of signals.
3. **Additive synthesis.** Additive implementation, monophony and polyphony.
4. **Modulation.** LFOs, envelopes, sequencers and drum machines, arpeggiation, modulation matrices, MIDI modulation.
5. **Subtractive synthesis.** Oscillators, antialiasing and Nyquist, wave shaping, wave folding, phase distortion, combination and amplification.
6. **Digital filters.** Transfer functions, poles and zeros, magnitude and phase response, Laplace domain, Z domain and the Bilinear transform, second-order filters, filter composition, first-order filters, fourth-order ladder filters, formants.
7. **Frequency Modulation synthesis.** Phase modulation, sidebands, Bessel functions and reflection, operators and algorithms.
8. **Sampling.** Pulse code modulation, wavetable synthesis, granular synthesis, resampling techniques, real-time interpolation.
9. **Effects.** Delays, flangers, chorus, reverb, phasers, physical modeling synthesis.
10. **Controllers.** MIDI protocols.
11. **The Fourier Transform.** DFT and FFT, windowing, STFT.

Synthesizers for Sound Design

https://www.youtube.com/watch?v=NJLIS2MkFe4

- Go beyond simulating the sounds of an acoustic instrument
- Widely used in EDM



Figure from: https://professionalcomposers.com/synth-pad-quick-guide/



Sound Design and Synth Fundamentals

Sample-based Synthesis

- **Vocaloid** by Xavier Serra
  https://en.wikipedia.org/wiki/Xavier_Serra

Daisy was a collaboration project with Yamaha. The aim of the project was to develop a singing voice synthesizer in which the user would input the lyrics and the notes of a vocal melody and obtain a synthetic performance of a virtual singer. To synthesize such performance the system concatenates a chain of elemental synthesis units. These units are obtained by transposing and time-scaling samples from singers databases. These databases are created out of recording, analyzing, labeling and storing singers performing in as many different musical and phonetic contexts as possible.

Based on Daisy's research, Yamaha released a product named Vocaloid. Vocaloid was presented at the 114th Audio Engineering Society (AES) Convention in Amsterdam in March 2003 and had a big media impact leaded by The New York Times article "Could I Get That Song in Elvis, Please?". The Vocaloid synthesizer was Nominee for the European IST (Information Society Technologies) Prize 2005 and is the choice of leading artists including Mike Oldfield.

The research carried out for Daisy is mainly described in Bonada and Loscos (2003), Bonada et al. (2003), and Bonada et al (2001).



Source:
Alex Loscos, *Spectral Processing of the Singing Voice*, PhD Thesis, UPF, Spain (supervised by Xavier Serra)

Sample-based Synthesis: Ample Sound

http://www.amplesound.net/en/index.asp

- Challenges:
  - Great **manual effort** in recording
    - Every note, velocity, playing technique, etc
  - **Note-note transition**



Sample-based Synthesis: Ample Sound

https://x.com/Reo5419233/status/1853347613116072218

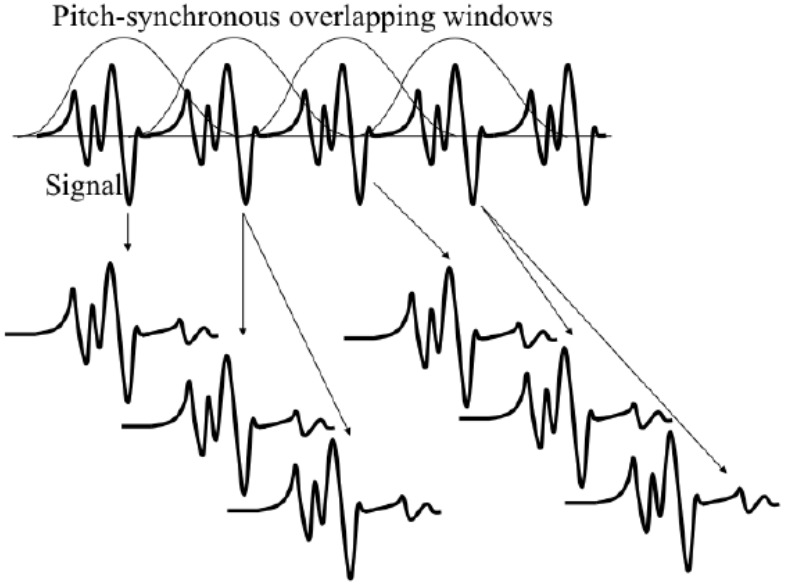


Overlap-and-Add for Audio Time Stretching and Pitch Scaling

https://en.wikipedia.org/wiki/Audio_time_stretching_and_pitch_scaling

https://en.wikipedia.org/wiki/PSOLA

- **PSOLA**: Pitch synchronous overlap-and-add
  - Divide the waveform into small overlapping segments
  - To **change the pitch**, the segments are *moved further apart* (to decrease the pitch) or *closer* together (to increase the pitch)
  - To **change the duration**, the segments are then *repeated* multiple times (to increase the duration) or some are *eliminated* (to decrease the duration)



Ref: Figure from https://www.researchgate.net/figure/Pitch-Synchronous-Overlap-Add-PSOLA_fig2_264006670

Audio Signal Processing by PyTSMod

(Audio Time Stretching and Pitch Scaling)

https://seyong92.github.io/PyTSMod-ISMIR2020LBD/

Available TSM algorithms

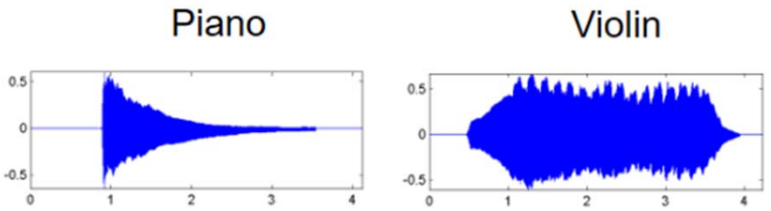
- Overlap-Add (OLA)
  - OLA is the most simple TSM algorithm that changes the length of the signal through modifying the hop size between analysis frame and synthesis frame.
- Pitch Synchronous Overlap-Add (TD-PSOLA)
  - TD-PSOLA is the algorithm that analyzes the orignal waveforms to create pitich-synchronous analysis windows and synthesize the output signal both for modifying time-scale and pitch-scale.
- Waveform Similarity Overlap-Add (WSOLA)
  - WSOLA maximizes the waveform similarity by allowing timing tolerance to analysis frame to find the most similar position through cross correlation.
- Phase Vocoder (PV)
  - Phase vocoder estimates instantaneous frequency, and it is used to update phases of input signal's frequency components in short-time Fourier transform. Although TSM results with phase vocoder has high phase continuity, it causes an transient smearing for percussive audio sources and a coloring artifact called phasiness.
- TSM based on harmonic-percussive source separation (HPTSM)
  - A novel TSM algorithm that applying phase vocoder to only harmonic sources and applying OLA to only percussive sources.

Outline

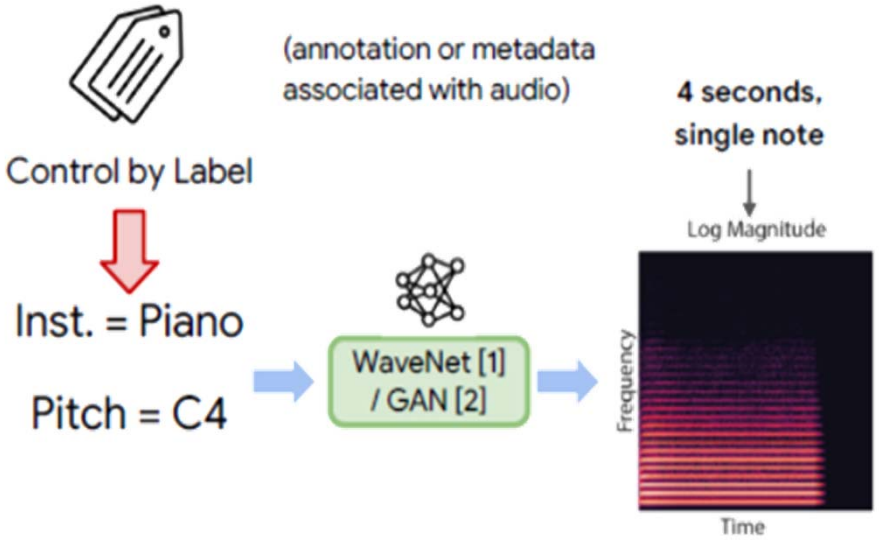
- DSP-based music synthesizers
- **DL-based generation of single notes**
  - WaveNet Autoencoder
  - GANSynth
  - InstrumentGen
- DL-based generation of note sequences
- MIDI-to-audio

Neural Generation of Single Notes

- "**MIDI pitch → audio**"
- Condition on pitch and instrument
- Modeling mainly **timbre**



- Can we learn such a mapping in a *data-driven* way via DL?



Ref: https://github.com/lukewys/ISMIR2022-tutorial

Autoregressive Models for Unconditional Audio Generation

- Generate **raw waveforms**
- **Unconditional** generation (not music synthesis yet)
  - **WaveNet** (Van Den Oord et al., 2016)
  - SampleRNN (Mehri et al., 2016)
  - WaveRNN (Kalchbrenner et al., 2018)
  - Inference with these models is inherently slow and inefficient because audio samples must be generated sequentially

WaveNet (arXiv'16)

In this paper we introduce a new generative model operating directly on the raw audio waveform. The joint probability of a waveform $\mathbf{x} = \{x_1, \ldots, x_T\}$ is factorised as a product of conditional probabilities as follows:

$$p(\mathbf{x}) = \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}) \quad (1)$$

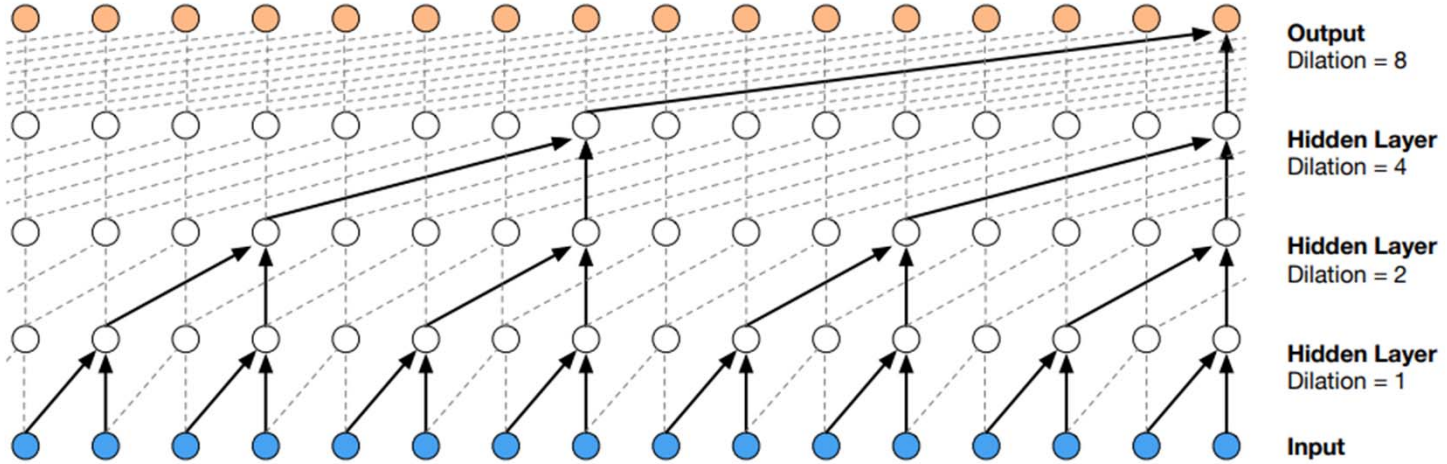
Each audio sample x_t is therefore conditioned on the samples at all previous timesteps.

- **Autoregressive; causal** convolutions
  - The prediction at timestep *t* cannot depend on any of the future timesteps
  - "At training time, the conditional predictions for all timesteps can be made in **parallel** because all timesteps of ground truth x are known"
  - "When generating with the model, the predictions are **sequential**: after each sample is predicted, it is fed back into the network to predict the next sample"

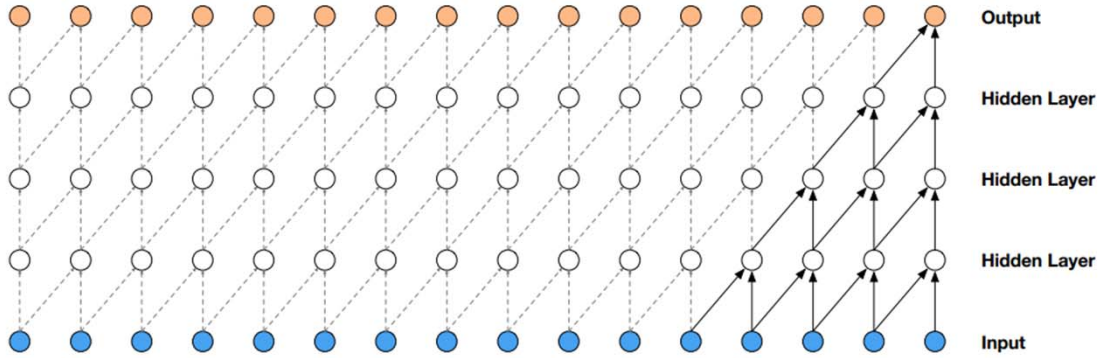
Ref: van den Oord et al, "WaveNet: A generative model for raw audio," arXiv 2016

WaveNet: Dilated Convolutions

- *Convolution with holes*
  - "A convolution where the filter is applied over an area larger than its length by skipping input values with a certain step"
  - Exponentially increasing the dilation factor results in exponential receptive field growth with depth



Usual convolution



WaveNet AutoEncoder (ICML'17)

- Extension of WaveNet for "**MIDI pitch → audio**"
  - Encoder-decoder; conditional generation
- **WaveNet Autoencoder**
  - WaveNet-like **encoder** that infers hidden embeddings $Z = f(x)$ distributed in time
  - WaveNet **decoder** that uses Z to reconstruct the original audio autoregressively

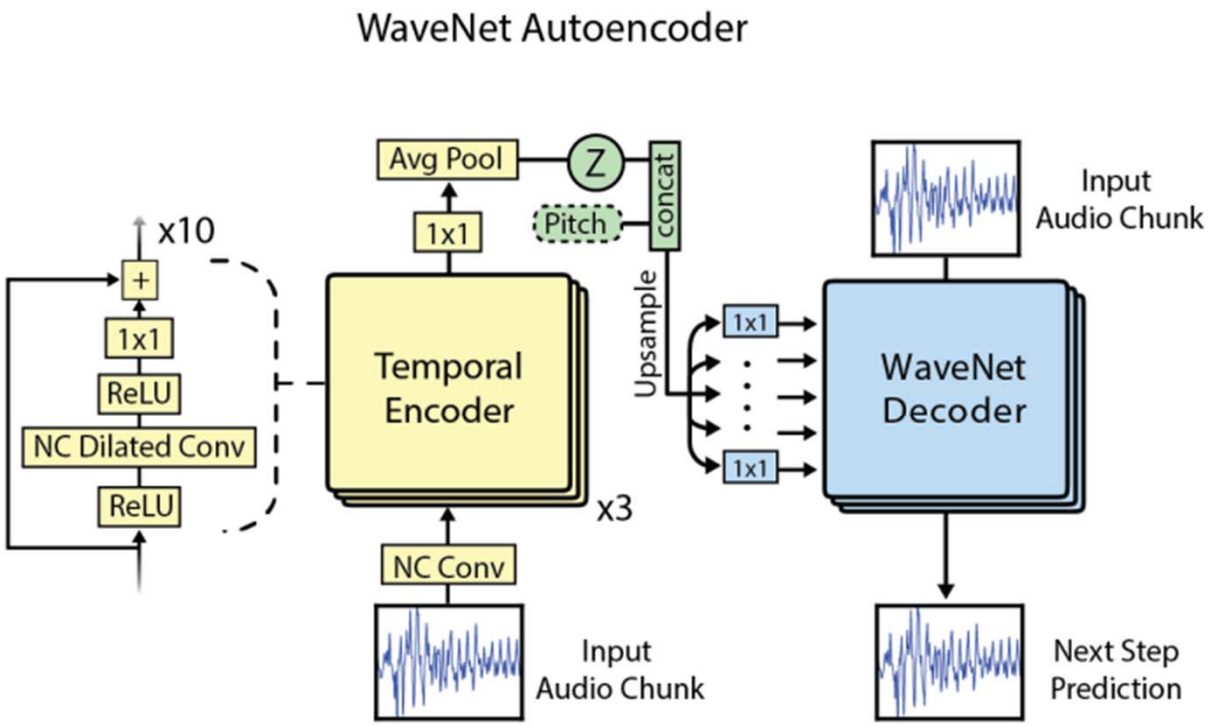
$$p(x) = \prod_{i=1}^{N} p(x_i | x_1, ..., x_{N-1}, f(x))$$

  - At inference time, the embedding can be inferred deterministically from audio or drawn from other points in the embedding space, e.g., through interpolation

Ref: Engel et al, "Neural audio synthesis of musical notes with WaveNet autoencoders," ICML 2017

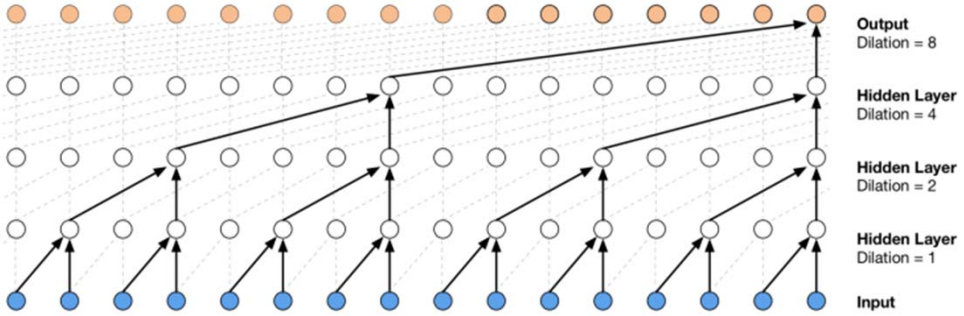
WaveNet AutoEncoder

- *WaveNet-like encoder*: **non-causal** (NC)**, non-autoregressive**
- *WaveNet decoder*: **causal, autoregressive**

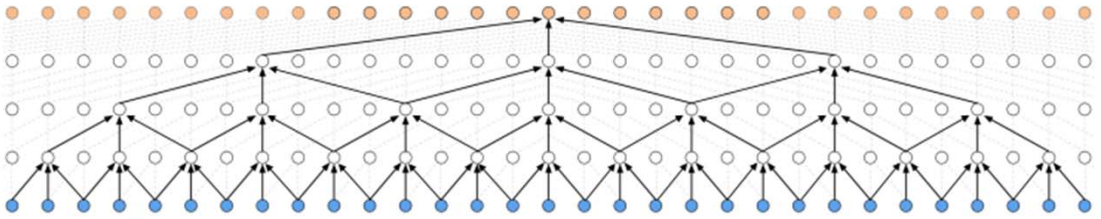


WaveNet AutoEncoder

- *WaveNet-like encoder*: **non-causal** (NC)**, non-autoregressive**
- *WaveNet decoder*: **causal, autoregressive**



a stack of dilated causal convolutional layers



a stack of dilated non-causal convolutional layers

Dataset: NSynth

https://magenta.tensorflow.org/nsynth

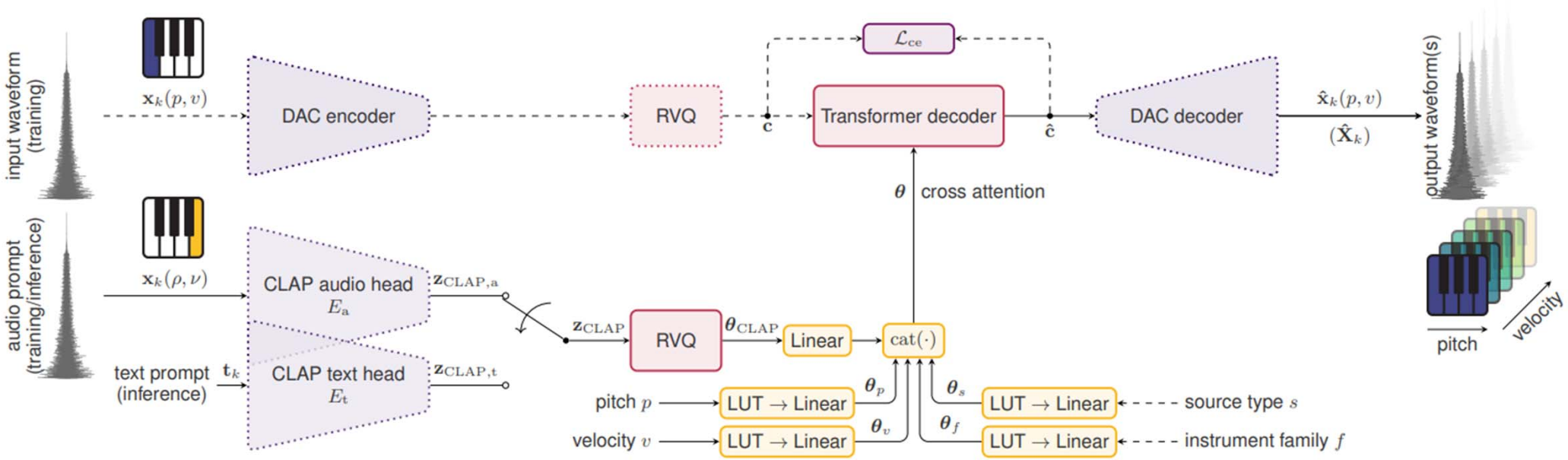
- Across a range of pitches, timbres, and volumes

	Source			
Family	Acoust	Electr	Synth	**Total**
Bass	200	8387	60 368	68 955
Brass	13 760	70	0	13 830
Flute	6572	70	2816	9458
Guitar	13 343	16 805	5275	35 423
Keyboard	8508	42 709	3838	55 055
Mallet	27 722	5581	1763	35 066
Organ	176	36 401	0	36 577
Reed	14 262	76	528	14 866
String	20 510	84	0	20 594
Synth Lead	0	0	5501	5501
Vocal	3925	140	6688	10 753
Total	108 978	110 224	86 777	306 043

InstrumentGen

https://instrumentgen.netlify.app/

- Take **pitch**, **velocity**, and **text** as input



Ref: Nercessian et al, "Generating sample-based musical instruments using neural audio codec language models," ISMIR 2024

InstrumentGen

https://instrumentgen.netlify.app/

Advanced Descriptive Prompts/Limitations

To showcase the capabilities and current limitations of the model in response to advanced descriptive prompts, we present the following examples.

Select MIDI velocity: 100

Prompt	Generated audio (2 octaves)
A string ensemble characterized by high harmonics, light bowing, and sparse vibrato, yielding an airy, floating tonal quality.	0:00 / 1:40
A bass synth with a distorted sawtooth waveform and high resonance, delivering a gritty, aggressive sonic texture.	0:00 / 1:40
Staccato piano notes augmented with a synthetic overlay and digital delay, producing a crisp, rhythmically precise tonal effect.	0:00 / 1:40

Sample-to-Instrument

Although it is not the central focus of our paper, our system inherently accommodates a sample-to-instrument functionality, whereby a musical instrument can be generated from a single audio reference as input. We notionally demonstrate this with an out-of-domain audio sample used as the prompt.

Prompt	Generated audio (2 octaves)
0:00 / 0:01	0:00 / 1:40

Ref: Nercessian et al, “Generating sample-based musical instruments using neural audio codec language models,” ISMIR 2024

Outline

- DSP-based music synthesizers
- DL-based generation of single notes
- **DL-based generation of note sequences**
  - Deep Performer
  - MIDI-DDSP
- MIDI-to-audio

Phrase Generation: Sequence of MIDI Pitches → Audio

- Note-to-audio
  - "Single MIDI pitch → audio"
  - Like a **synthesizer**, or an "**instrument**"
  - Let the human musician play it

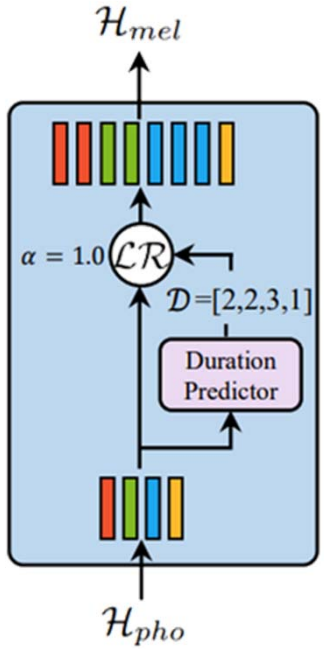


- Score-to-audio
  - "Sequence of MIDI pitches → audio"
  - More autonomous
  - Generate timbre and **expressive performance**
    - Need to account for **note-note transition**, position of a note in a phrase, etc

Monophonic Phrase Generation

- We have actually learned this before, in **singing voice synthesis** (SVS), for **monophonic** note sequence
  - Conversion between **symbolic timing** to **physical timing** via mechanisms such as duration predictor
  - Use neural networks to learn note-note transitions, and the way to sing each note depending on their location in a phrase

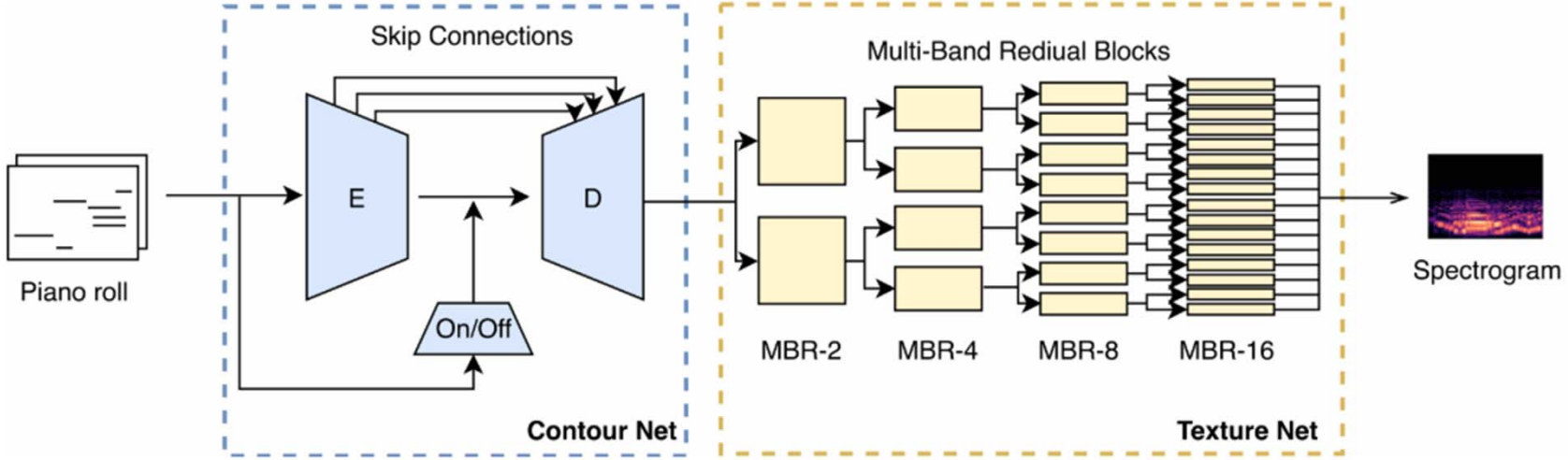




- How about the **polyphonic** case?

Polyphonic Phrase Generation

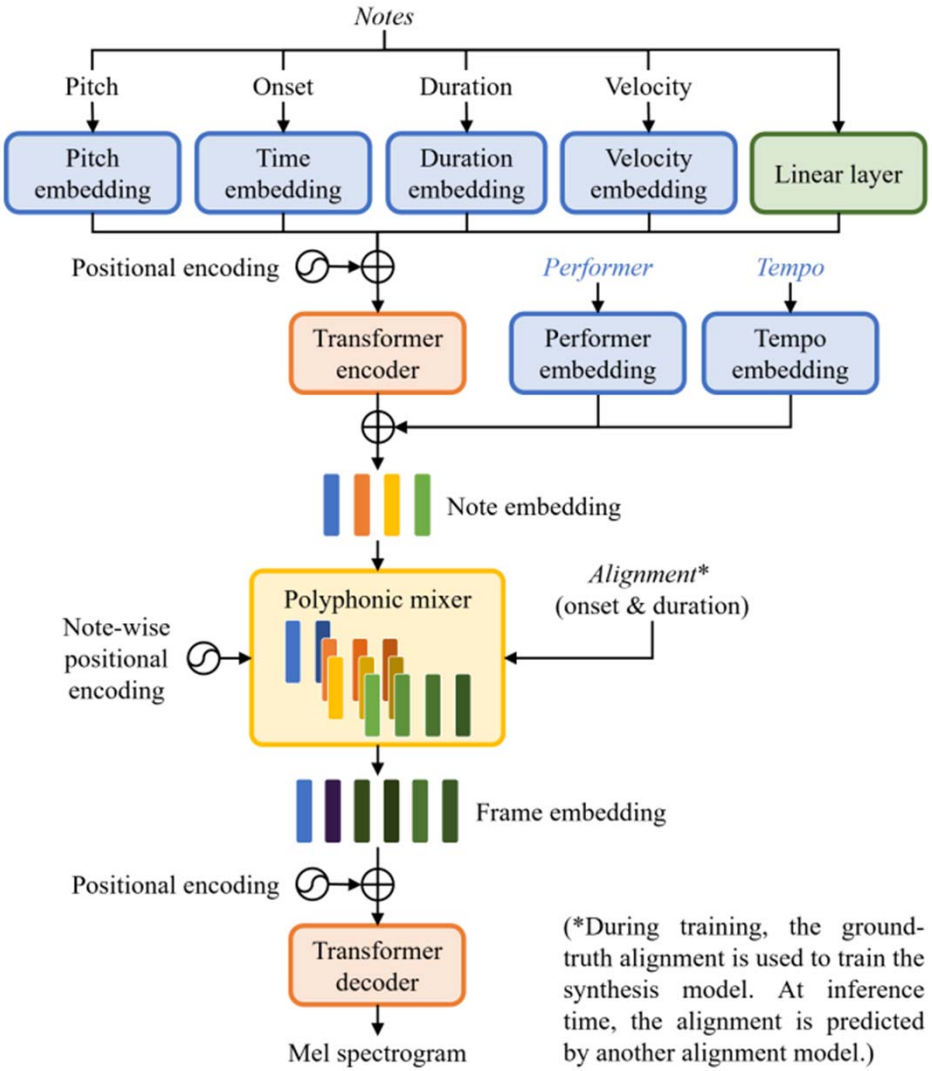
- **PerformanceNet**
  - **CNN** approach converting pianorolls (MIDI performances) into Mel spectrograms
  - Limits: assume the input and output are of the same length (no need to care about symbolic-to-physical timing conversion)



Ref: Wang & Yang, "PerformanceNet: Score-to-audio music generation with multi-band convolutional residual network," AAAI 2018

Polyphonic Phrase Generation

- **Deep Performer**
  - **Transformer**-based approach being inspired by **SVS** models
  - Use an **alignment** module for symbolic-to-physical timing conversion, by predicting the **onset** and **duration** of each note
    - NOTE: onset prediction is not needed for monophonic cases such as SVS
  - Use a **"polyphonic mixer"** to handle polyphonic inputs
    - sum up the note embeddings for the same frame according to onsets, durations



Ref: Dong et al, "Deep Performer: Score-to-audio music performance synthesis," ICASSP 2022

Deep Performer

https://hermandong.com/deepperformer/



Fig. 4: Examples of the alignments predicted by (a) the constant-tempo baseline model and (b) Deep Performer, our proposed model. (c) shows the input score.

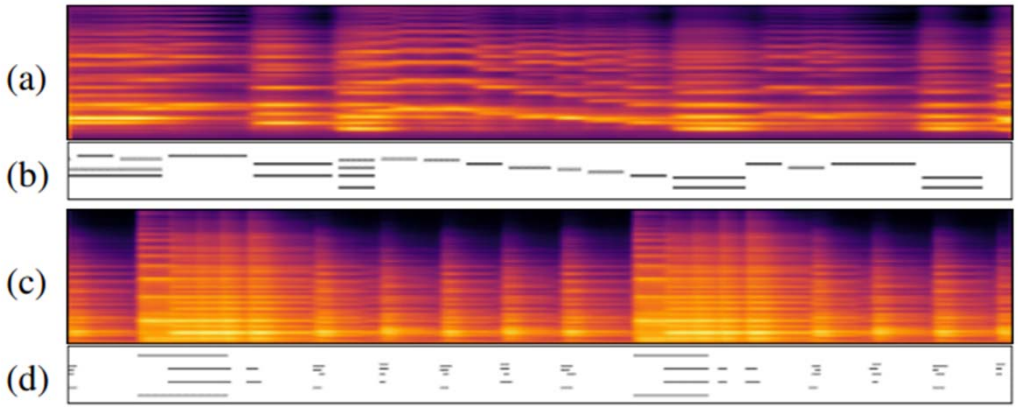


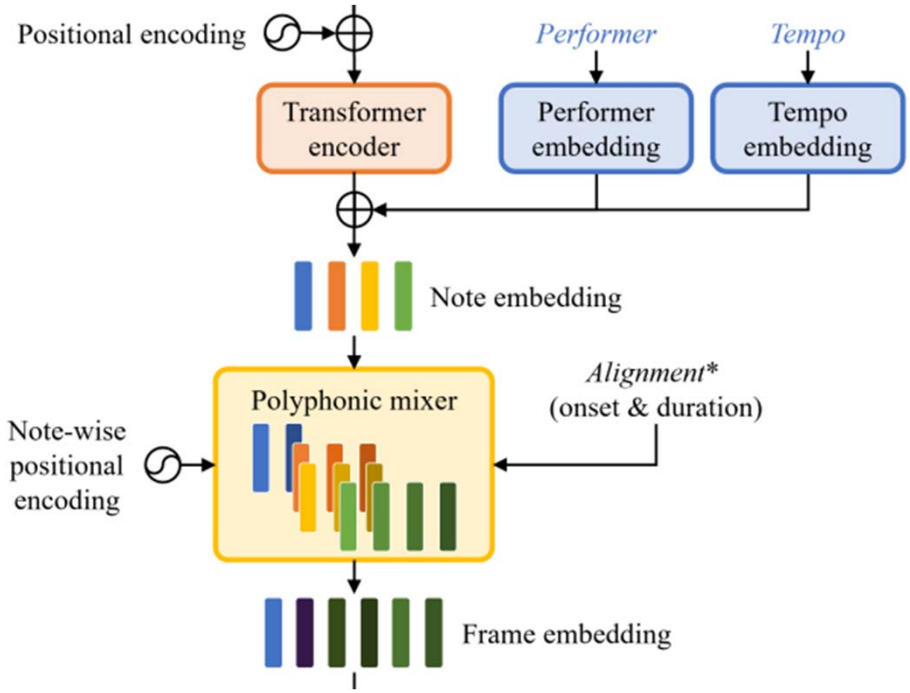
Fig. 5: Examples of the mel spectrograms, in log scale, synthesized by our proposed model for (a) violin and (c) piano. (b) and (d) show the input scores for (a) and (c), respectively.

- Future work mentioned
  - Use articulation marks and ornaments on scores to model playing techniques
  - disentangle timbre from room acoustics
  - incorporate adversarial losses

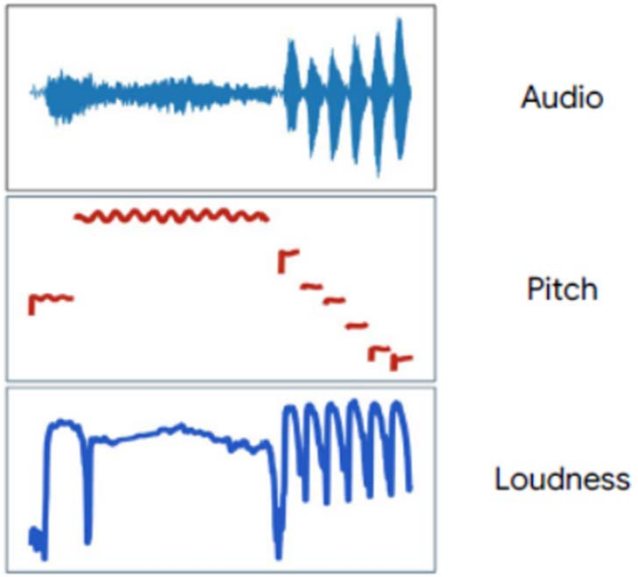
Ref: Dong et al, "Deep Performer: Score-to-audio music performance synthesis," ICASSP 2022

Controllability of MIDI-to-Audio Generation

- Deep Performer offers control of **onset** and **duration**

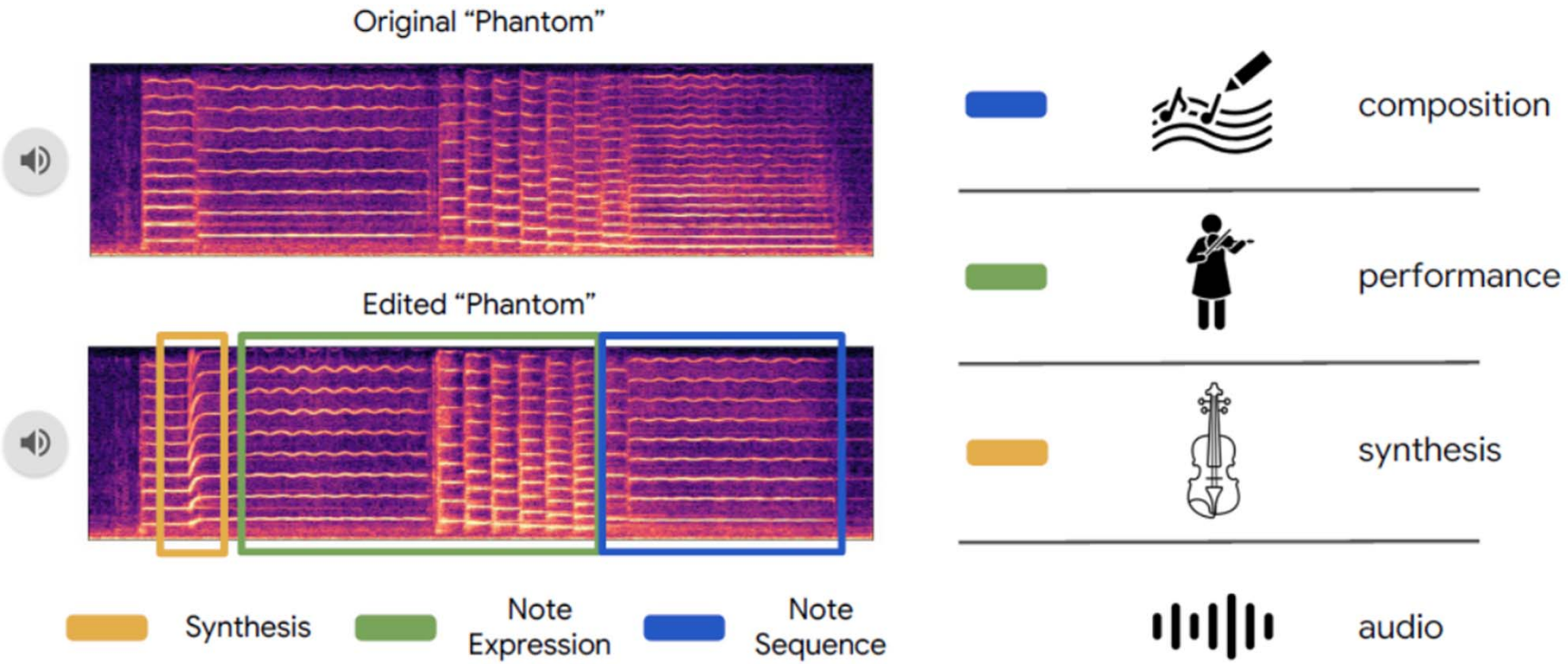


- But there are other **low-level, time-varying** quantities that we may want to control
- E.g., **F0 contour**, **loudness**



MIDI-DDSP: Controlling Instrument Synthesis

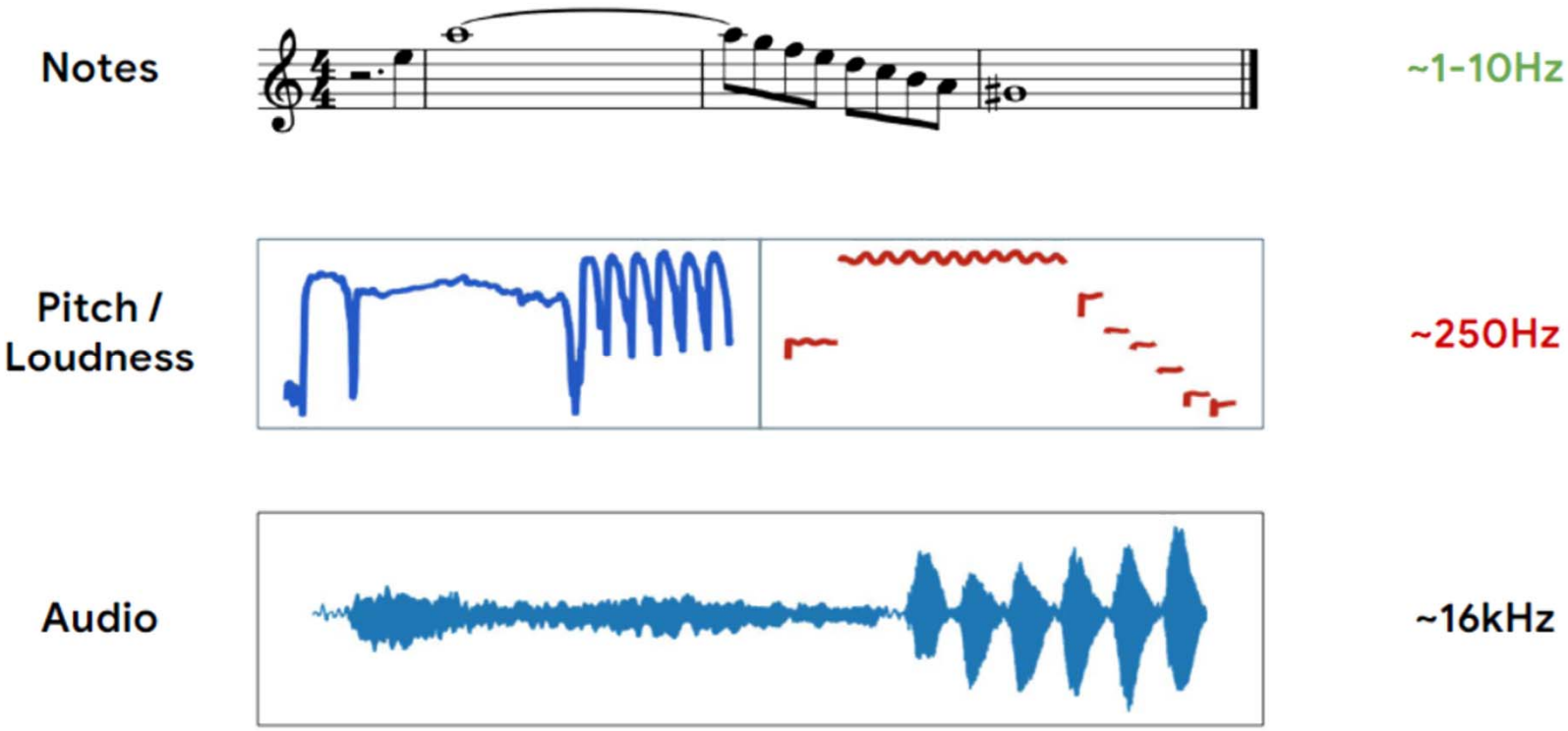
https://youtu.be/7U-zDL5con8?si=HcD7YDN66YPlyGCN&t=9783



Ref: Wu et al, "MIDI-DDSP: Detailed control of musical performance via hierarchical modeling," ICLR 2022

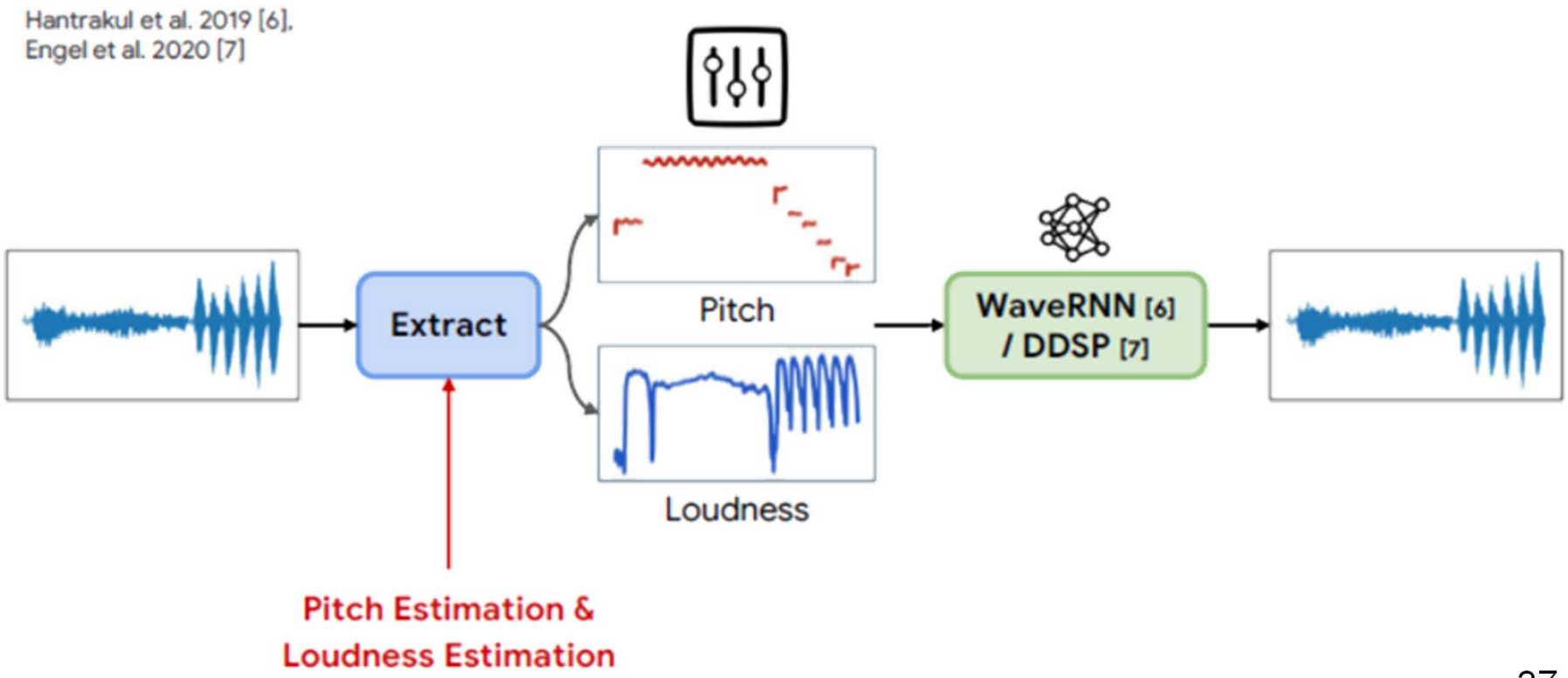
Low-level Quantities

https://github.com/lukewys/ISMIR2022-tutorial



Extract the Label: Pitch and Loudness

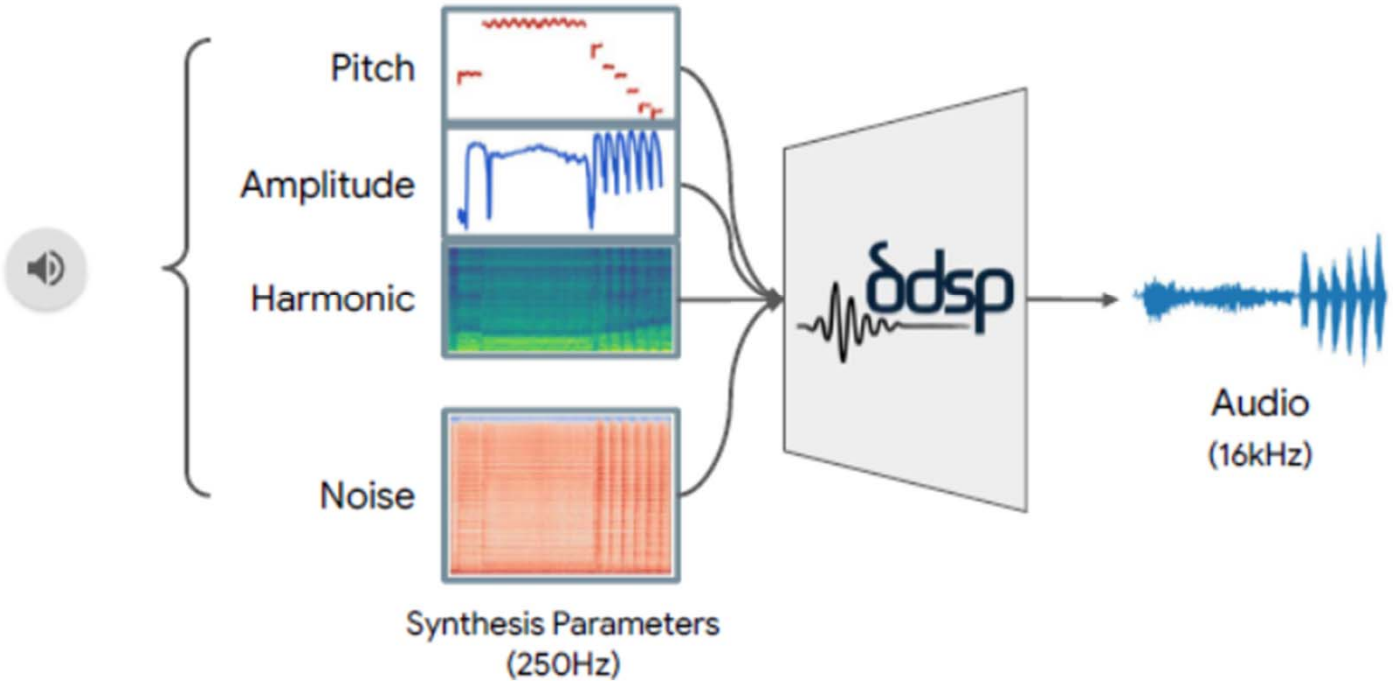
https://github.com/lukewys/ISMIR2022-tutorial



Learn to Extract Synthesis Parameters: DDSP

https://github.com/lukewys/ISMIR2022-tutorial

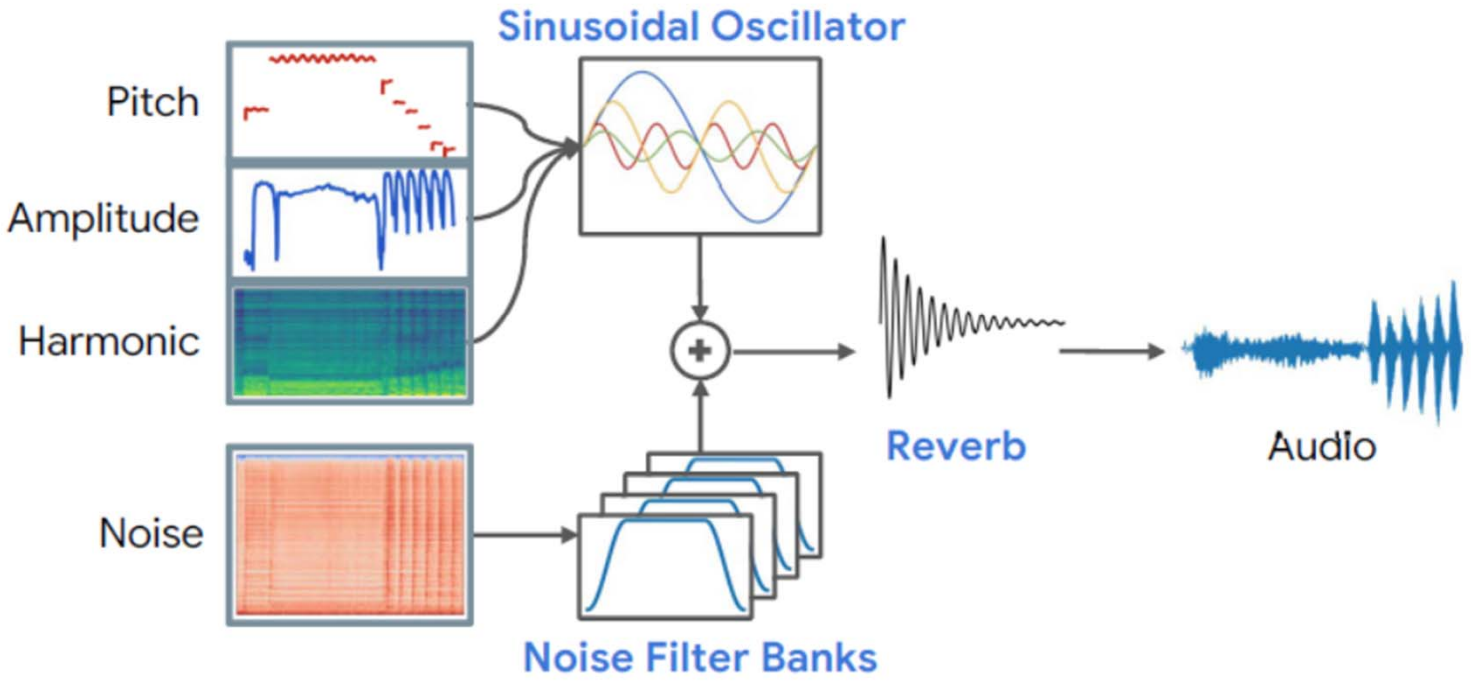
Engel et al. 2020 [7]



DDSP: Differentiable Digital Signal Processing

https://github.com/lukewys/ISMIR2022-tutorial

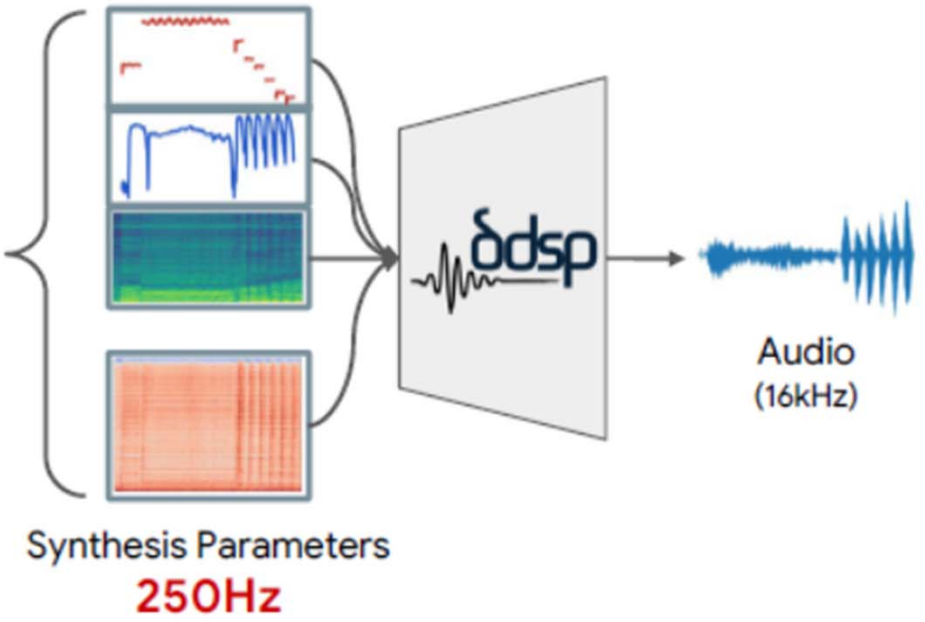
Engel et al. 2020 [7]



Problem of Low-level Control

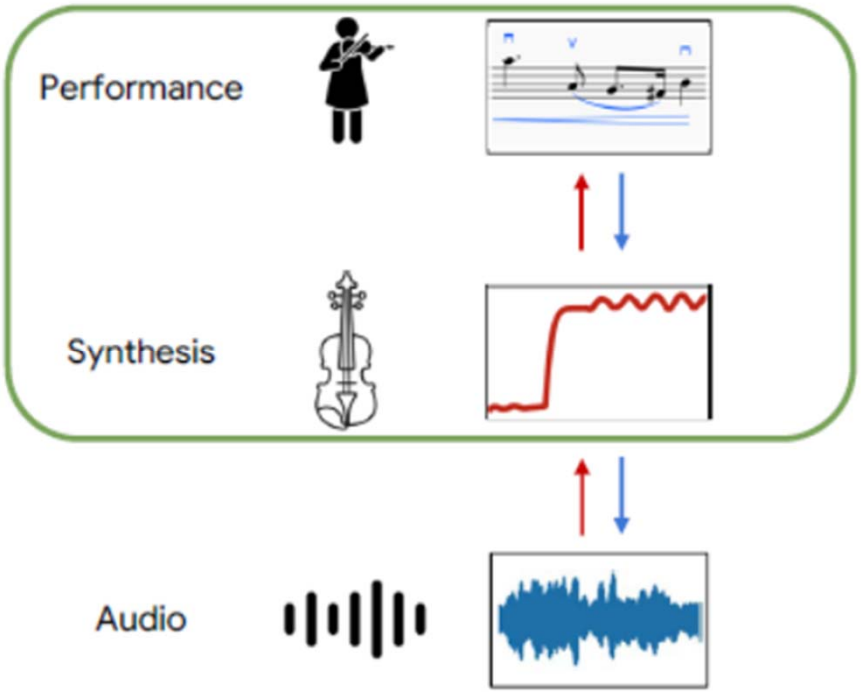
https://github.com/lukewys/ISMIR2022-tutorial

1 sec: **127 x 250 = ~32k**
parameters to control!!
Need high-level Control



Extract Performance Parameter

https://github.com/lukewys/ISMIR2022-tutorial

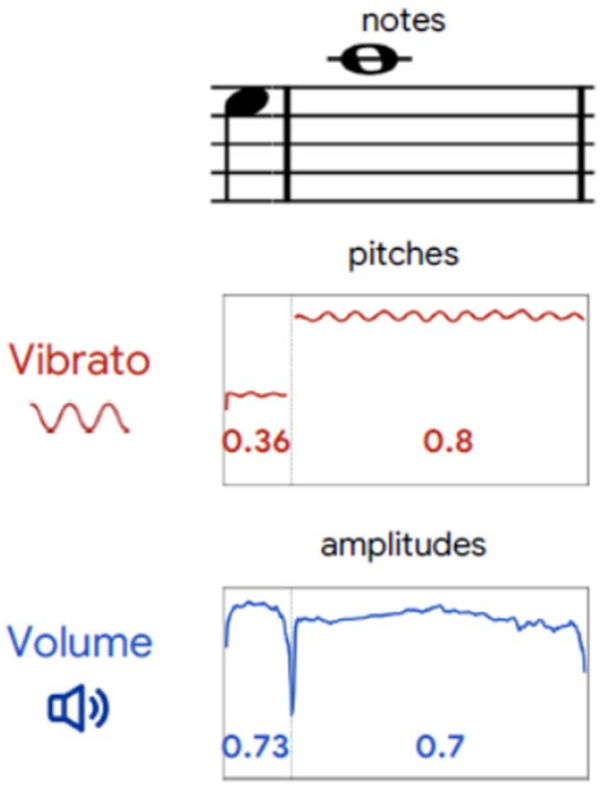


Summary statistics pooled over notes
6-D **scalar features**, scaled [0,1]:

- Volume
- Vibrato
- Brightness
- Attack Noise
- Volume Peak Position
- Volume Fluctuation

Extract Performance Parameter

https://github.com/lukewys/ISMIR2022-tutorial

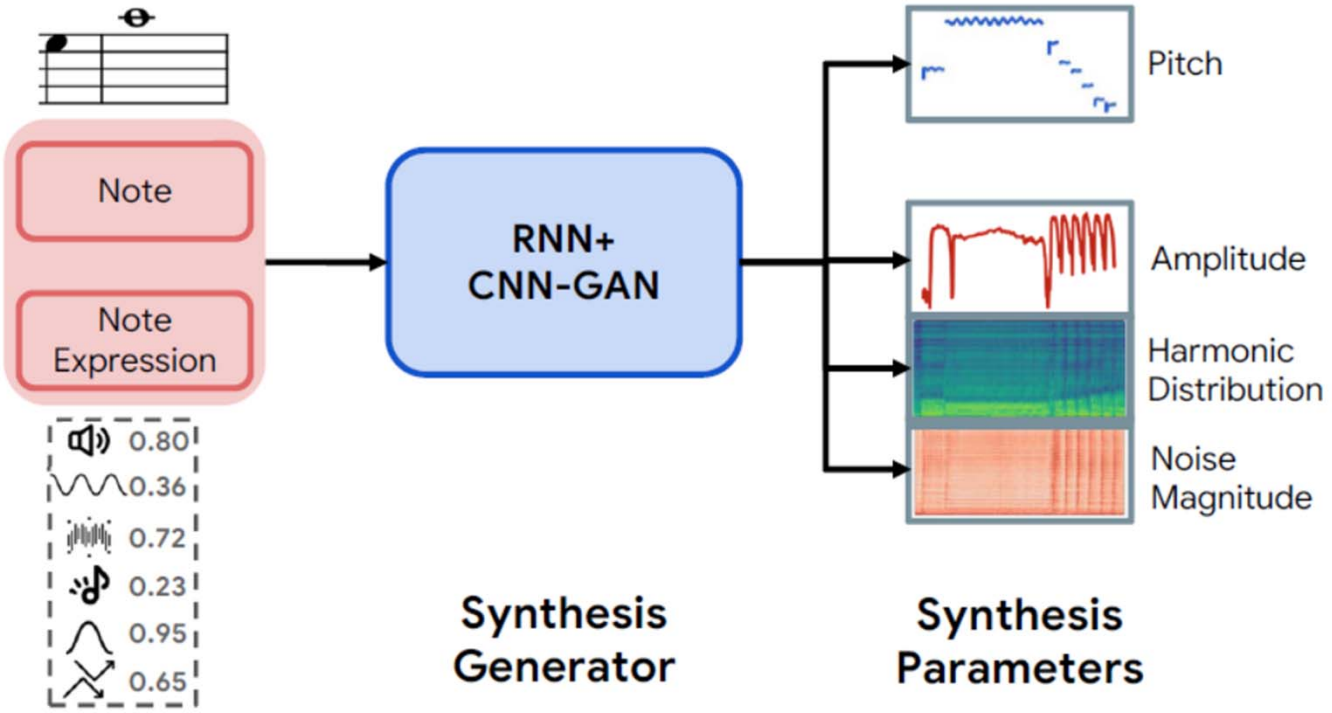


Summary statistics pooled over notes
6-D **scalar features**, scaled [0,1]:

- Volume
- Vibrato
- Brightness
- Attack Noise
- Volume Peak Position
- Volume Fluctuation

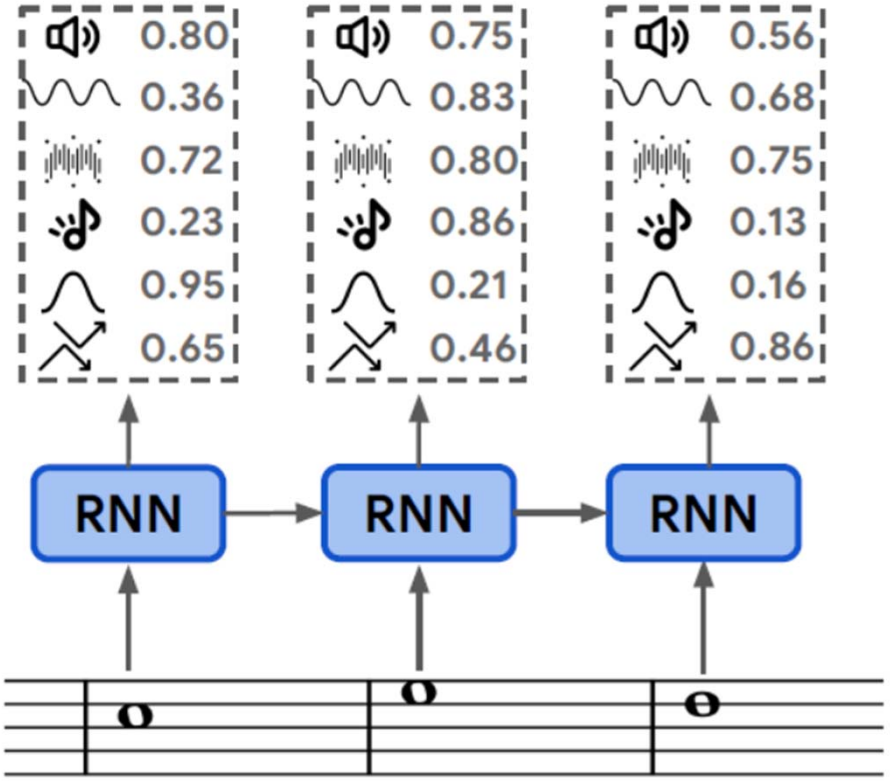
Synthesis Generator

https://github.com/lukewys/ISMIR2022-tutorial



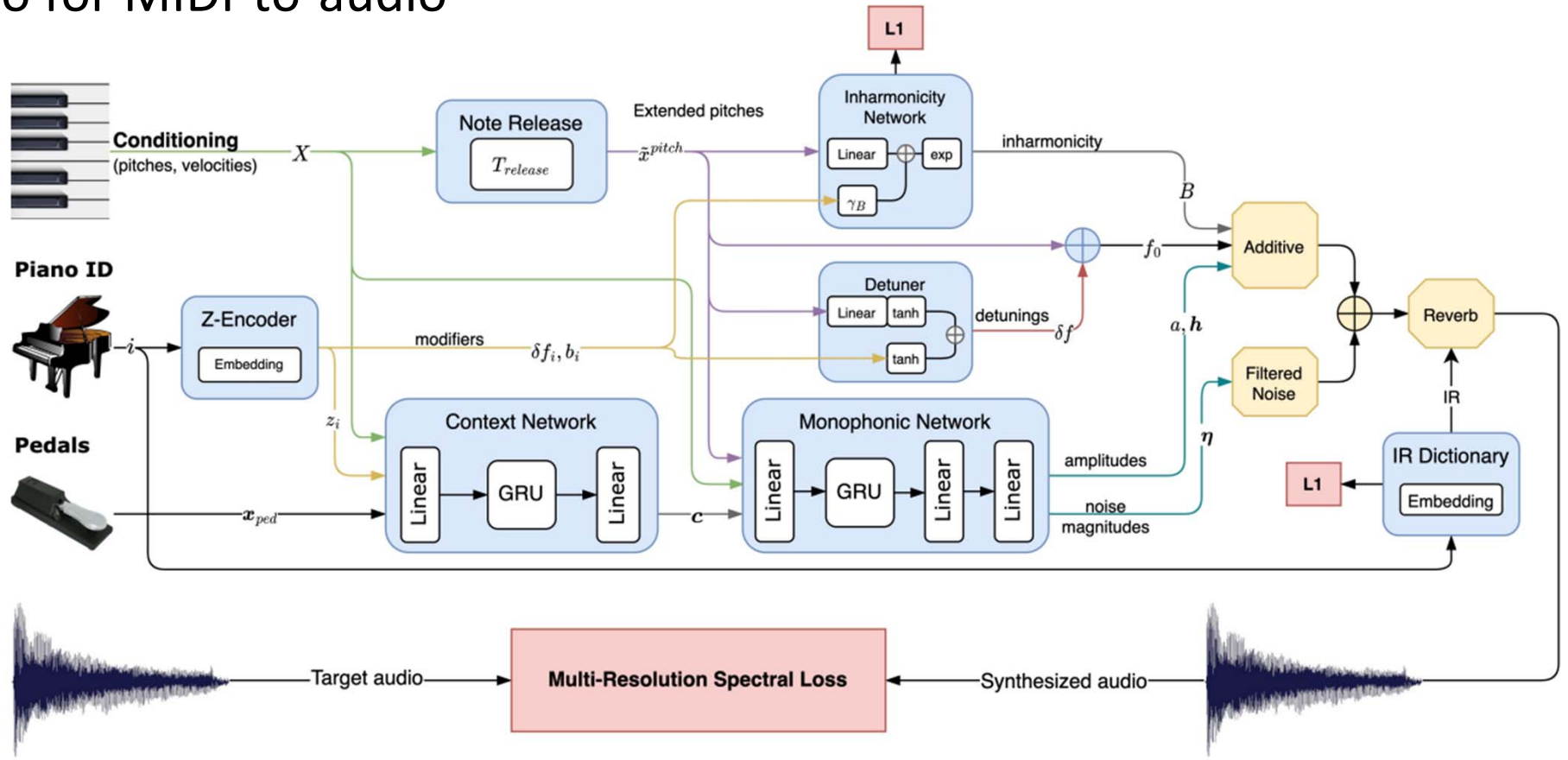
Autoregressive Prior on Expression Controls

https://github.com/lukewys/ISMIR2022-tutorial



DDSP-Piano

- Also for MIDI-to-audio

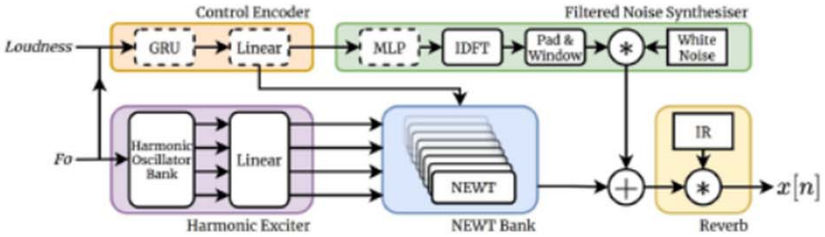


Ref: Renault et al, “DDSP-Piano: a neural sound synthesizer informed by instrument knowledge,” AES 2023

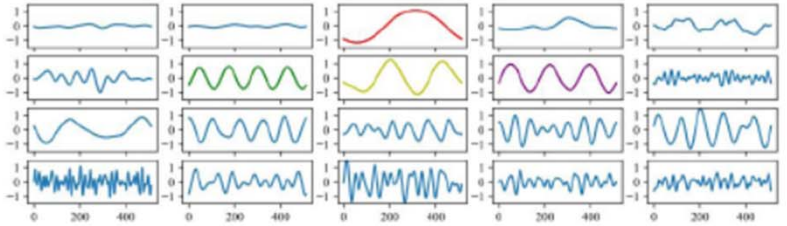
Other Differentiable Synthesis Works

- Also for MIDI-to-audio

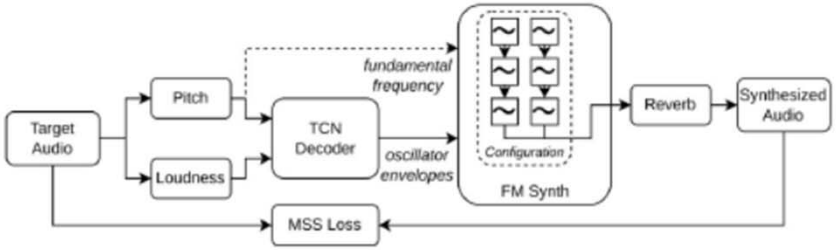
Waveshaping Synthesis [8]



Wavetable Synthesis [9]



FM Synthesis [10]

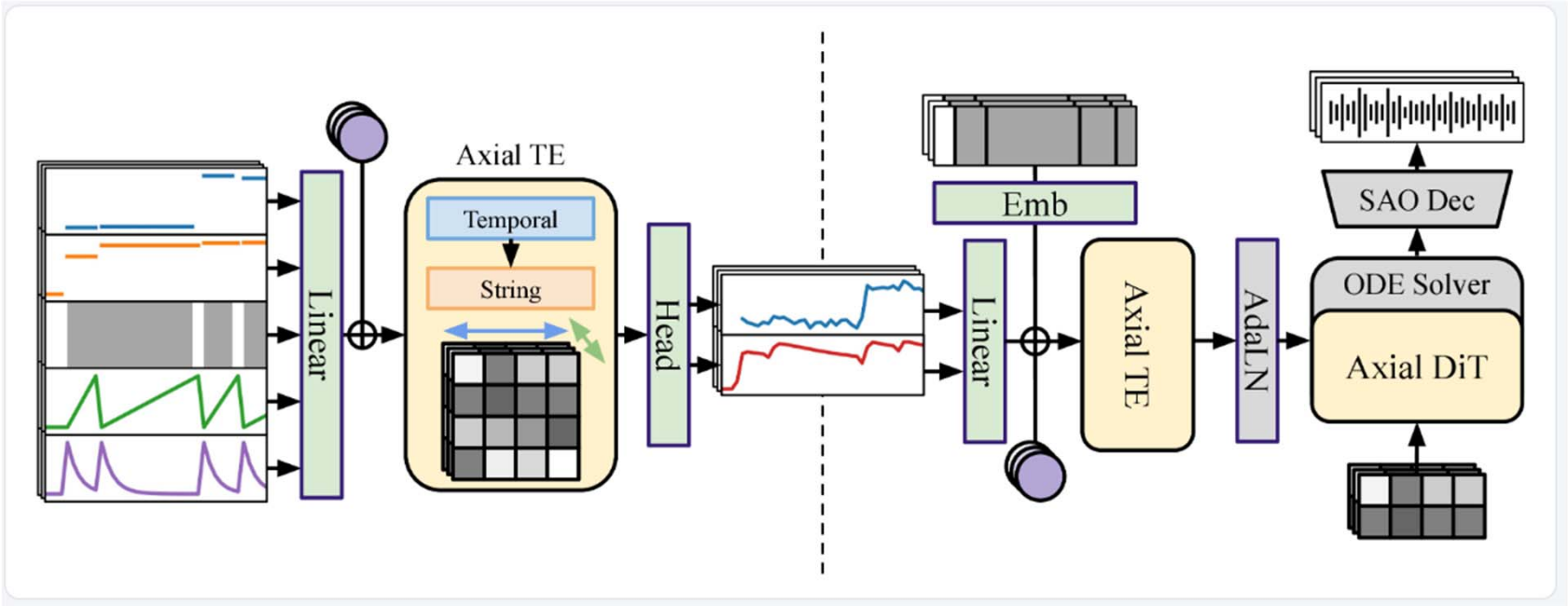


Ref: https://github.com/lukewys/ISMIR2022-tutorial

Expressive Guitar Synthesis via Controllable Latent Flow Matching

https://weifu3024.github.io/ctrl-lfm-guitar-synth/

- "String-wise" MIDI → f0 and amplitude control → SAO latents → audio



Ref: Fu et al, "Expressive guitar synthesis via controllable latent flow matching," arXiv 2026

Outline

- DSP-based music synthesizers
- DL-based generation of single notes
- DL-based generation of note sequences
- **MIDI-to-audio**

MIDI-to-Audio

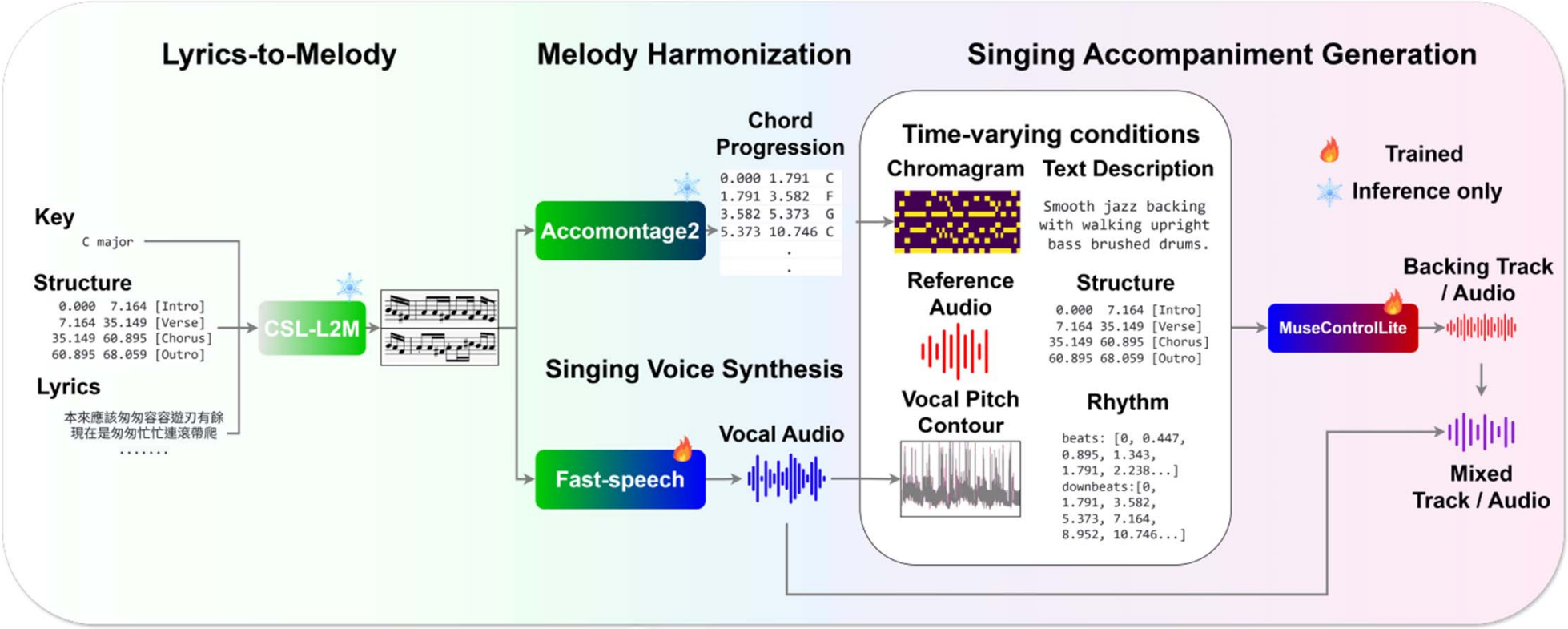
- **text → leadsheet → audio**: MIDI-SAG, YuE2
- **text → MIDI → audio**: ongoing work

- **piano MIDI → pop audio:** Piano2Pop
- **multi-track MIDI → audio:** ongoing work

- **text → (MIDI) → audio**: MIDI-REPA
- **text → MIDI + audio**: MuseDuo

Text → Leadsheet → Audio: MIDI-SAG

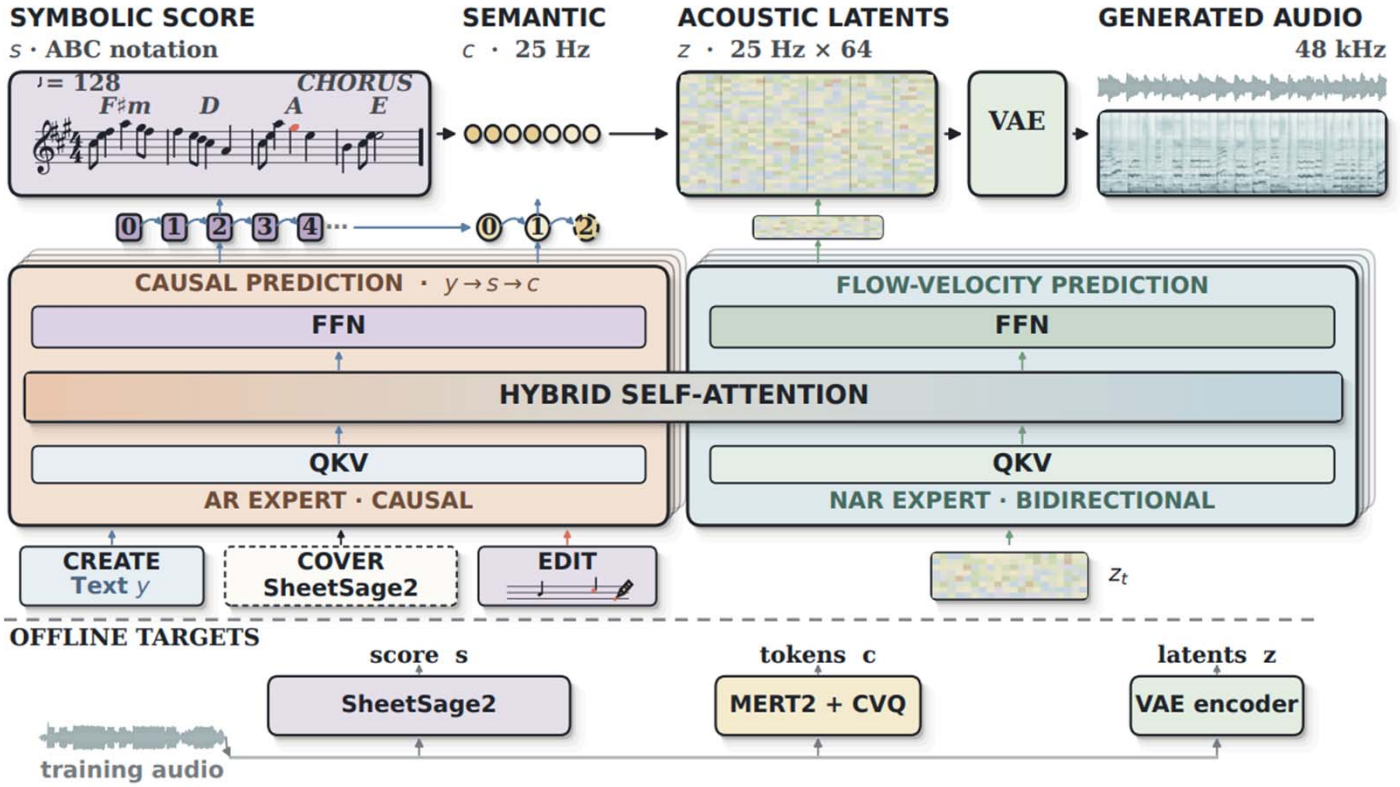
https://composerflow.github.io/web/



Ref: Tsai et al, "MIDI-informed singing accompaniment generation in a compositional song pipeline," arXiv **2026. 02**

Text → Leadsheet → Audio: YuE2

https://github.com/multimodal-art-projection/YuE



Ref: Yuan et al, "YuE2: Unifying symbolic and audio music generation at frontier quality," arXiv **2026.09**

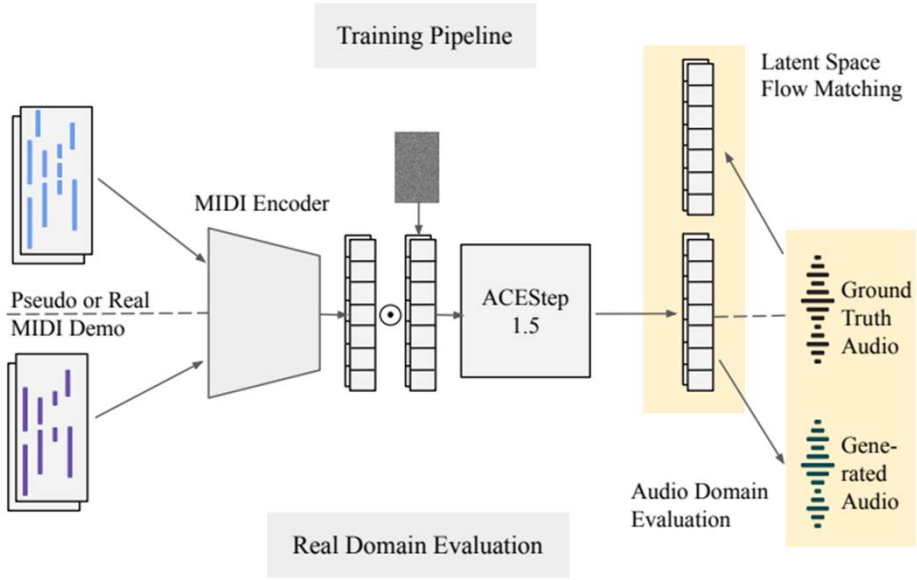
Text → MIDI → Audio

- Text→MIDI: MIDI-LLM

Piano MIDI → Pop Audio: Piano2Pop

https://hhhhaura.github.io/piano2pop/

- Motivation: “demo” to song
- Inversed version of Pop2Piano & PicoGen
- **Pianoroll encoder + AceStep 1.5 DiT**
- Data
  - Training: “transcribed” MIDI → audio
  - Inference: real piano MIDI→ audio
- Data augmentation to bridge the gap between transcribed MIDI and real MIDI



Ref: Cheng et al, “Piano2Pop: Expanding piano demos into pop productions without aligned supervision” arXiv 2026

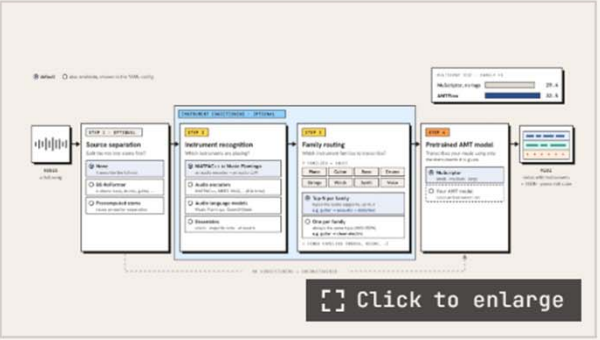
Multi-track MIDI → Audio

- Training data: “transcribed” MIDI → audio

Text → (MIDI) → Audio: MIDI-REPA

https://midi-repa.vaclis.net/

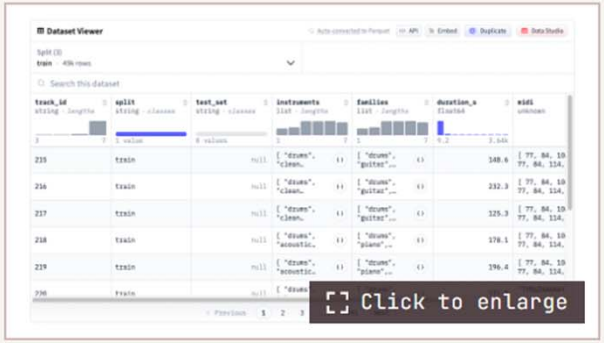
- Use automatic music transcription (AMT) to derive MIDI supervision
- Then align them with the intermediate features of a flow-matching DiT



PIPELINE

AMTFlow ↗

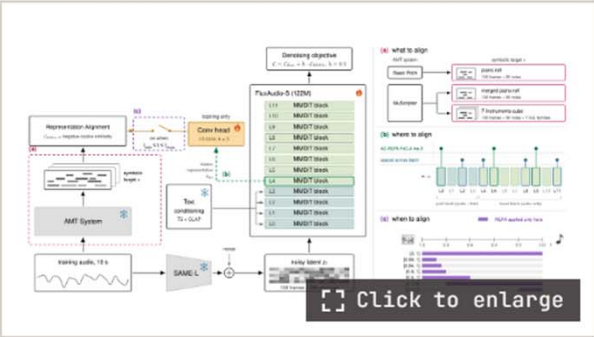
Transcription pipeline used for the paper. To reproduce, run with `--config paper`.



DATASET

JamendoMIDI ↗

Multi-instrument MIDI transcriptions of 51,077 instrumental Jamendo tracks. CC BY-NC-SA 4.0.



CODE

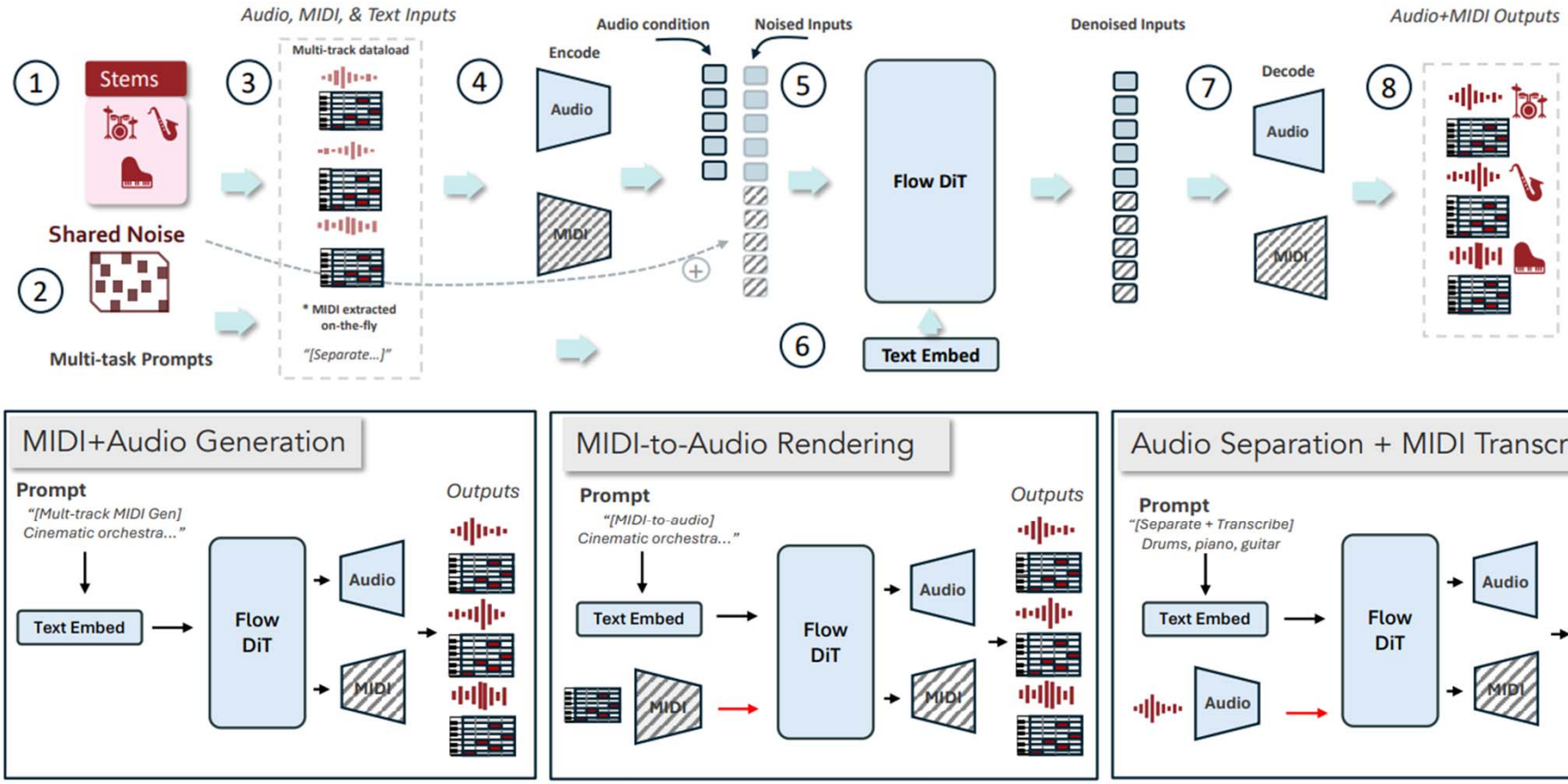
MIDI-REPA ↗

Full training implementation and recipe for MIDI-REPA.

Ref: Yu et al, “MIDI-REPA: Improving text-to-music generation with symbolic representation alignment,” arXiv 2026

Text → MIDI + Audio: MuseDuo

https://museduo-demo.vercel.app/



Ref: MuseDuo: Unified audio-MIDI music generation, separation, transcription, rendering, and editing," ICLR 2027 submission